



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Aplicación de Tecnologías Internet



Laboratorio #5

Alumnos:

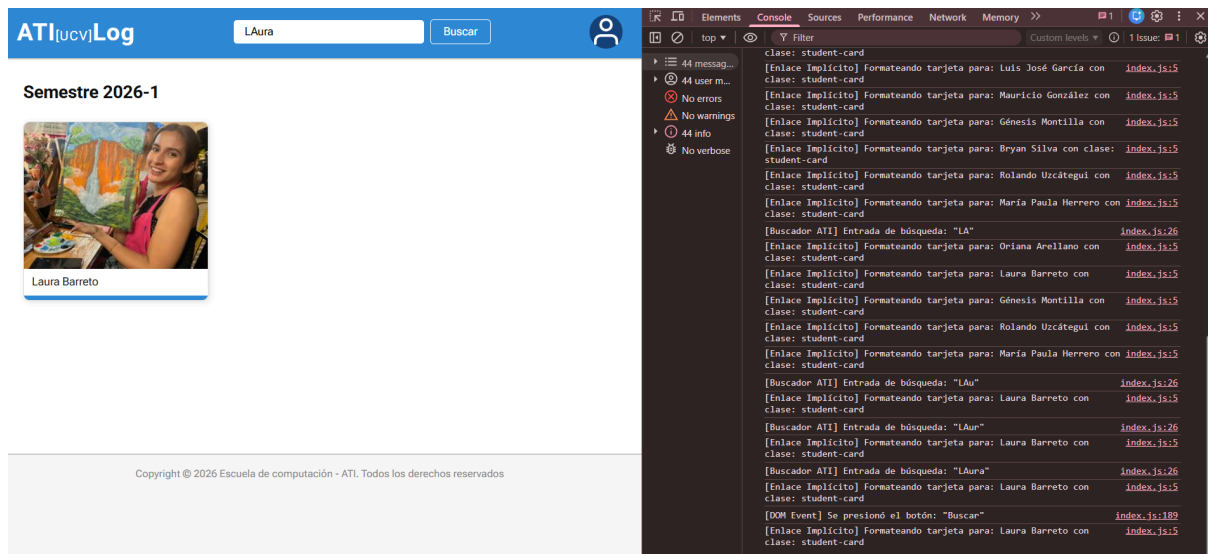
Laura Barreto

CI:

27279497

Caracas, Mayo 2026.

PARTE 1



Aca podemos ver 3 funciones que tienen el this:

- Función 1:

Enlace Implícito (Implicit Binding)

Código: `this.cardClass` (líneas 5 y 8).

Contexto: Dentro del método `formatCard` de la constante `profileFormatter`.

Explicación: Como la función se invoca directamente precedida por un objeto con un punto (`profileFormatter.formatCard(...)`), `this` se enlaza automáticamente a dicho objeto (`profileFormatter`). Por ello, `this.cardClass` accede a la propiedad del objeto ("`student-card`").

```
const profileFormatter = {
  cardClass: "student-card",
  formatCard: function (profile, lang) {
    console.log(`[Enlace Implícito] Formateando tarjeta para:
    ${profile.name} con clase: ${this.cardClass}`);
    const card = document.createElement("a");
    card.className = this.cardClass;

    let cardHref = `profile.html?ci=${profile.ci}`;
    if (lang) {
      cardHref += `&lang=${lang}`;
    }
    card.href = cardHref;
```

```

        return card;
    }
};

```

- Función 2

Enlace Léxico (Lexical Binding)

Código: `this.logLabel` (línea 26).

Contexto: Dentro de una función flecha (arrow function) que actúa como callback del evento `input`.

Explicación: Las funciones flecha no definen su propio `this`, sino que lo heredan de su ámbito contenedor inmediato. Como la función flecha se declaró dentro de la función normal `track`, hereda el `this` de `track` (el cual apunta al objeto contenedor `searchTracker`). Esto permite acceder a `this.logLabel` ("`[Buscador ATI]`") en lugar de apuntar al elemento `<input>`.

```

const searchTracker = {
  logLabel: "[Buscador ATI]",
  track: function (inputElement) {
    if (inputElement) {
      inputElement.addEventListener("input", (e) => {
        const query = e.target.value.trim();
        console.log(`${this.logLabel} Entrada de búsqueda:
"${query}"`);
        renderProfiles(query);
      });
    }
  }
};

```

- Función 3

Enlace en Manejador de Eventos DOM (DOM Event Binding)

Código: `this.textContent` (línea 188).

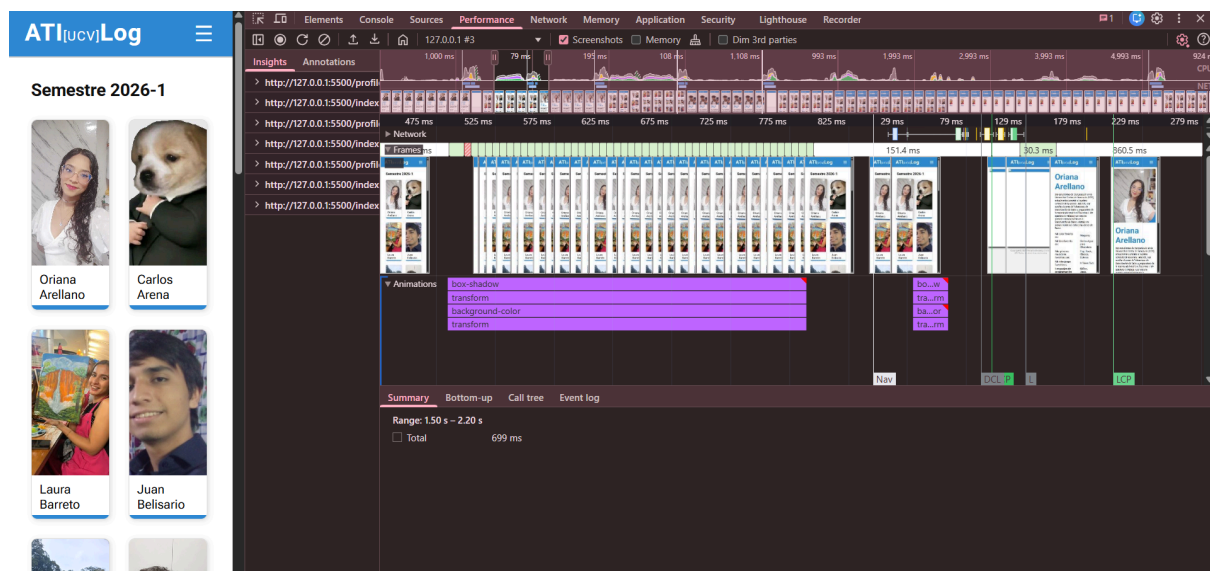
Contexto: Dentro de una función callback tradicional (function) asociada al evento `click` del botón de buscar.

Explicación: Al utilizar la palabra clave `function` (función clásica) en un manejador de eventos del navegador, la API del DOM enlaza dinámicamente `this` con el elemento HTML que disparó el evento (`HTMLButtonElement`). De esta forma, `this.textContent` extrae dinámicamente el texto del botón del DOM ("`Buscar`").

```
const searchButton = document.querySelector(".nav-search button");
if (searchButton && searchInput) {
  searchButton.addEventListener("click", function () {
    const query = searchInput.value.trim();
    console.log(`[DOM Event] Se presionó el botón: "${this.textContent}"`);
    renderProfiles(query);
  });
}
```

PARTE 2

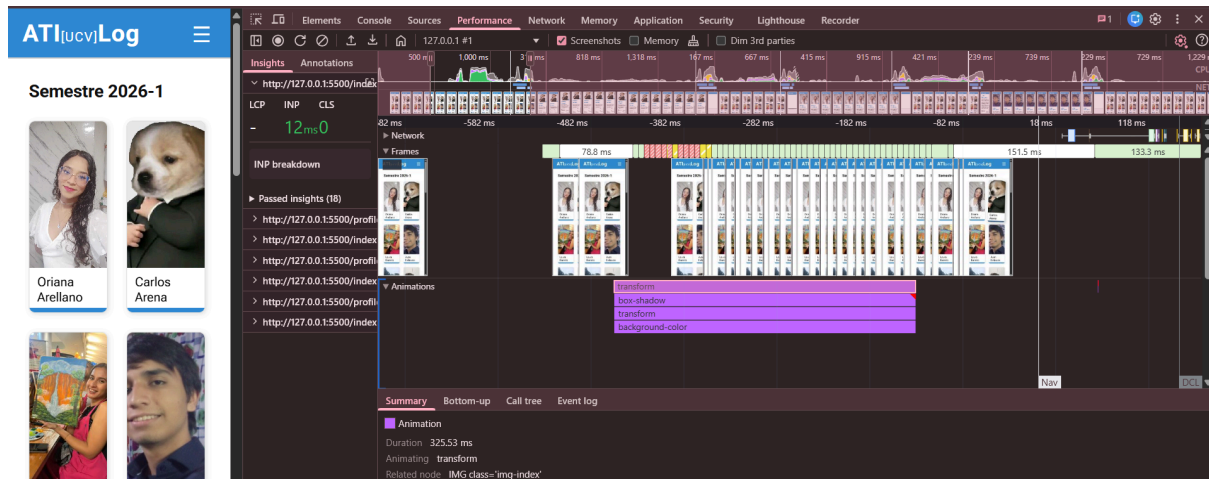
Identifica los selectores que consumen más tiempo realizando el análisis con DevTools.



Los selectores que consumen más serían:

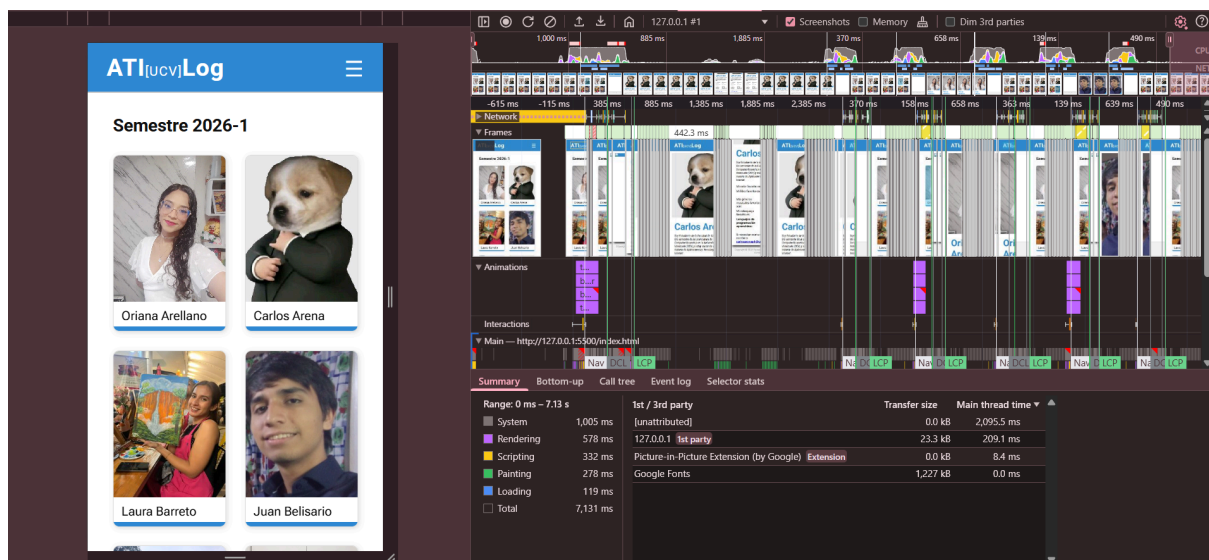
- background-color
- transform
- box-shadow

Si agregamos mejoras, tenemos que:



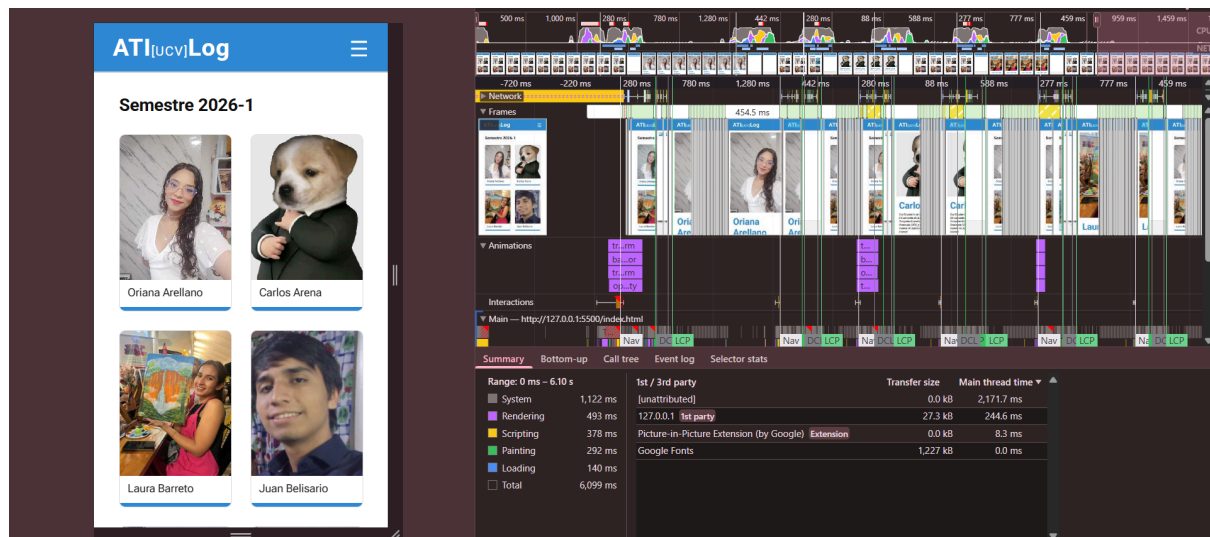
Se optimizó el rendimiento reemplazando el selector universal y los descendientes costosos por clases directas y CSS Custom Properties, eliminando así el recorrido innecesario del DOM. Adicionalmente, se promovieron los elementos clave a capas de composición aceleradas por hardware mediante el uso estratégico de la propiedad `will-change` antes del `hover`.

PARTE 3



Reducimos la velocidad para dispositivos de menor rendimiento a CPU: 4x - slowdown

Podemos ver que hay un cuello de botella en el hilo principal por un exceso de Rendering y System, causado por activar demasiadas animaciones juntas al interactuar en las tarjetas de los estudiantes (.student-card). Esto sobrecarga al navegador y hace que la página dé tirones graves, generando congelamientos de hasta 442.3 ms.



Con el cambio, se reemplazó la transición de box-shadow (causaba repaint en cada frame) por un pseudo-elemento `::after` que anima solo opacity, propiedad 100% compositable en GPU sin repaint. Se añadió debounce(220ms) al buscador para que el DOM no se reconstruya en cada tecla, se usó DocumentFragment para insertar todas las tarjetas en un solo reflow en vez de N, y se agregó `loading="lazy"` a las imágenes para no cargarlas todas al mismo tiempo.

